

module-02-flow-control

Chapter 2: Flow Control | 37 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). PPT Module-1: boolean, comparison, if/else/elif, while, break/continue, for/range, import, sys.exit

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Flow control decides WHICH lines run. A condition is an expression that is True or False. An if/while/for statement ends with a colon and controls an indented block. Indentation is not decoration in Python - it IS the structure. while repeats while True; for repeats a fixed set from range(); break exits now, continue skips to the next round.

Key ideas to carry forward

- Comparison operators (==, !=, <, >, <=, >=) evaluate to True/False. Never confuse == with =.
- and/or/not combine Booleans: True and False -> False; not True -> False.
- if runs once, elif chains choices, else catches everything left over.
- while spam < 5: repeats until false - always change the variable inside or it loops forever.
- for i in range(5): gives 0..4; range(12,16) gives 12..15; range(0,10,2) steps by 2.
- import random, sys, os, math then use random.randint(1,10); sys.exit() ends early.

Classic beginner mistakes

- ! IndentationError from mixing tabs/spaces or dedenting wrongly.
- ! while True with no break, or forgetting i += 1 (infinite loop).
- ! range(5) stops at 4 - the end value is excluded.

Study order hint: Skim 01-09 for Boolean logic, 10-16 for if/elif/else, 17-23 for while/break/continue, 24-33 for for/range, 34-37 for imports.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. == vs = vs is - explain.

A: = stores a value; == compares values for equality; is checks same object in memory. Beginners must never type = inside an if condition.

Q2. How do and/or/not short-circuit?

A: and stops at the first False; or stops at the first True. So (4 < 5) and (9 < 6) is False without needing more checks.

Q3. When do you pick while over for?

A: while when iterations are unknown (wait until input is valid); for/range when the count or collection is known.

Q4. What do break and continue do?

A: break exits the whole loop now; continue jumps to the next iteration's condition check.

Q5. What does range(0, 10, 2) produce?

A: 0, 2, 4, 6, 8 - start, stop-excluded, step. range(5) is 0..4.

Q6. Why does this loop forever: while spam < 5: print(spam)?

A: spam never changes inside the block, so the condition stays True. Always update the loop variable or break.

Teach it to others

The one analogy that sticks

Flow control is a railway junction: the condition is the signal, True sends the train down one track (the indented block), False sends it down the other (else). Loops are roundabouts - you exit only at your condition.

What your students will get wrong

- x The loop checks the condition after every line (only at the top of each round).
- x else belongs to the nearest if - indentation decides, not intention.
- x range(5) includes 5 (it stops at 4).

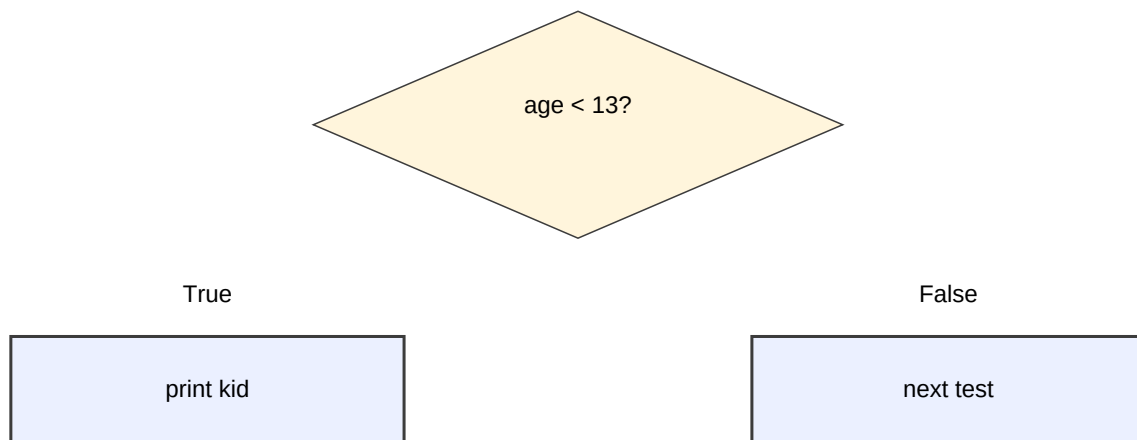
5-minute whiteboard plan

1. Draw the if/elif/else junction for 'vampire age' on the board.
2. Live-code a while name-check; deliberately forget the update, watch it hang, then Ctrl-C and fix.
3. Students predict range(12,16) and range(0,10,2) outputs before running.
4. Exit ticket: rewrite a while countdown as a for loop.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - if / elif / else is a railway junction



True takes the left track (indented block), False rolls to the next test. Every junction ends with a colon.

Visual 2 - while is a roundabout, for is a fixed track



Roundabout: you circle until the exit condition opens (update the variable!). Fixed track: exactly 5 stops, 0..4.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Vampire check (elif chain)

```
age = int(input('Age? '))
if age < 13:
    print('kid')
elif age < 20:
    print('teen')
elif age < 65:
    print('adult')
else:
    print('senior')
# EXPECTED: Age? 72 -> senior
```

X2. Password retries (while + break)

```
tries = 0
while True:
    pw = input('pw? ')
    tries += 1
    if pw == 'swordfish' or tries == 3:
        break
print('tries:', tries)
# EXPECTED: Correct on 2nd try -> tries: 2
```

X3. FizzBuzz slice (for + %)

```
for i in range(1, 16):
    if i % 15 == 0:
        print('FizzBuzz')
    elif i % 3 == 0:
        print('Fizz')
    elif i % 5 == 0:
        print('Buzz')
    else:
        print(i)
# EXPECTED: 3->Fizz, 5->Buzz, 15->FizzBuzz
```

Program map - all 37 at a glance

```
01 01_boolean_values.py
02 02_comparison_operators.py
03 03_comparison_operators_hello.py
04 04_comparison_operators_eggcount.py
05 05_binary_boolean_operators.py
06 06_binary_boolean_operators_false.py
07 07_not_operator.py
08 08_mixing_boolean_comparison.py
09 09_mixing_boolean_comparison_and.py
10 10_blocks_code.py
11 11_if_statements.py
12 12_else_statements.py
13 13_elif_statements.py
14 14_elif_statements_name.py
15 15_elif_statements_name_2.py
16 16_elif_statements_name_3.py
17 17_while_loop_statements.py
18 18_while_loop_statements_spam.py
19 19_while_loop_statements_name.py
20 20_while_loop_statements_al.py
21 21_break_statements.py
22 22_continue_statements.py
```

23 23_continue_statements_who.py
24 24_loops_range_function.py
25 25_loops_range_function_jimmy.py
26 26_loops_range_function_total.py
27 27_loops_range_function_my.py
28 28_loops_range_function_2.py
29 29_loops_range_function_3.py
30 30_loops_range_function_4.py
31 31_loops_range_function_5.py
32 32_loops_range_function_6.py
33 33_loops_range_function_7.py
34 34_importing_modules.py
35 35_importing_modules_2.py
36 36_importing_modules_random.py
37 37_ending_program_early.py

Topic: Boolean Values

01/37 01_boolean_values.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

OUT: ? >>> spam = True

spam

OUT: True

OUT: ? >>> true

OUT: Traceback (most recent call last):

OUT: File "<pyshell#2>", line 1, in <module>

OUT: true

OUT: NameError: name 'true' is not defined

OUT: ? >>> True = 2 + 2

OUT: SyntaxError: assignment to keyword

Topic: Comparison Operators

02/37 02_comparison_operators.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
42 == 42
```

```
# OUT:          True
```

```
42 == 99
```

```
# OUT:          False
```

```
2 != 3
```

```
# OUT:          True
```

```
2 != 2
```

```
# OUT:          False
```

03/37 03_comparison_operators_hello.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
'hello' == 'hello'  
# OUT: True  
'hello' == 'Hello'  
# OUT: False  
'dog' != 'cat'  
# OUT: True  
True == True  
# OUT: True  
True != False  
# OUT: True  
42 == 42.0  
# OUT: True  
42 == '42'  
# OUT: False
```

04/37 04_comparison_operators_eggcount.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
42 < 100  
# OUT:          True  
42 > 100  
# OUT:          False  
42 < 42  
# OUT:          False  
eggCount = 42  
eggCount <= 42  
# OUT:          True  
myAge = 29  
myAge >= 10  
# OUT:          True
```

Topic: Binary Boolean Operators

05/37 05_binary_boolean_operators.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

True and True

OUT: True

True and False

OUT: False

06/37 06_binary_boolean_operators_false.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
False or True
```

```
# OUT:          True
```

```
False or False
```

```
# OUT:          False
```

Topic: The not Operator

07/37 07_not_operator.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
not True
```

```
# OUT:          False
```

```
not not not not True
```

```
# OUT:          True
```

Topic: Mixing Boolean and Comparison Operators

08/37 08_mixing_boolean_comparison.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
(4 < 5) and (5 < 6)
```

```
# OUT:          True
```

```
(4 < 5) and (9 < 6)
```

```
# OUT:          False
```

```
(1 == 2) or (2 == 2)
```

```
# OUT:          True
```

09/37 09_mixing_boolean_comparison_and.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
2 + 2 == 4 and not 2 + 2 == 5 and 2 * 2 == 2 + 2
```

```
# OUT:          True
```

Topic: Blocks of Code

10/37 10_blocks_code.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Mary':  
    print('Hello Mary')  
    if password == 'swordfish':  
        print('Access granted.')  
    else:  
        print('Wrong password.')
```

Topic: if Statements

11/37 11_if_statements.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':  
    print('Hi, Alice.')
```

Topic: else Statements

12/37 12_else_statements.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':  
    print('Hi, Alice.')  
else:  
    print('Hello, stranger.')
```

Topic: elif Statements

13/37 13_elif_statements.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':  
    print('Hi, Alice.')  
elif age < 12:  
    print('You are not Alice, kiddo.')
```

14/37 14_elif_statements_name.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':  
    print('Hi, Alice.')  
elif age < 12:  
    print('You are not Alice, kiddo.')  
elif age > 2000:  
    print('Unlike you, Alice is not an undead, immortal vampire.')  
elif age > 100:  
    print('You are not Alice, grannie.')
```

15/37 15_elif_statements_name_2.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')
elif age > 100:
    print('You are not Alice, grannie.')
elif age > 2000:
    print('Unlike you, Alice is not an undead, immortal vampire.')
```

16/37 16_elif_statements_name_3.py [script]

Script: save and run; predict the output first, then compare.

```
if name == 'Alice':  
    print('Hi, Alice.')  
elif age < 12:  
    print('You are not Alice, kiddo.')  
else:  
    print('You are neither Alice nor a little kid.')
```

Topic: while Loop Statements

17/37 17_while_loop_statements.py [script]

Script: save and run; predict the output first, then compare.

```
spam = 0
if spam < 5:
    print('Hello, world.')
    spam = spam + 1
```

18/37 18_while_loop_statements_spam.py [script]

Script: save and run; predict the output first, then compare.

```
spam = 0
while spam < 5:
    print('Hello, world.')
    spam = spam + 1
```

19/37 19_while_loop_statements_name.py [script]

Script: save and run; predict the output first, then compare.

```
name = ''
while name != 'your name':
    print('Please type your name.')
    name = input()
print('Thank you!')
```

20/37 20_while_loop_statements_al.py [script]

Script: save and run; predict the output first, then compare.

```
Please type your name.  
Al  
Please type your name.  
Albert  
Please type your name.  
%#@#%*(^&!!!  
Please type your name.  
your name  
Thank you!
```

Topic: break Statements

21/37 21_break_statements.py [script]

Script: save and run; predict the output first, then compare.

```
while True:
    print('Please type your name.')
    name = input()
    if name == 'your name':
        break
print('Thank you!')
```

Topic: continue Statements

22/37 22_continue_statements.py [script]

Script: save and run; predict the output first, then compare.

```
while True:
    print('Who are you?')
    name = input()
    if name != 'Joe':
        continue
    print('Hello, Joe. What is the password? (It is a fish.)')
    password = input()
    if password == 'swordfish':
        break
print('Access granted.')
```

23/37 23_continue_statements_who.py [script]

Script: save and run; predict the output first, then compare.

```
Who are you?  
I'm fine, thanks. Who are you?  
Who are you?  
Joe  
Hello, Joe. What is the password? (It is a fish.)  
Mary  
Who are you?  
Joe  
Hello, Joe. What is the password? (It is a fish.)  
swordfish  
Access granted.
```

Topic: for Loops and the range() Function

24/37 24_loops_range_function.py [script]

Script: save and run; predict the output first, then compare.

```
print('My name is')  
for i in range(5):  
    print('Jimmy Five Times (' + str(i) + ')')
```

25/37 25_loops_range_function_jimmy.py [script]

Script: save and run; predict the output first, then compare.

```
My name is  
Jimmy Five Times (0)  
Jimmy Five Times (1)  
Jimmy Five Times (2)  
Jimmy Five Times (3)  
Jimmy Five Times (4)
```

26/37 26_loops_range_function_total.py [script]

Script: save and run; predict the output first, then compare.

```
total = 0
for num in range(101):
    total = total + num
print(total)
```

27/37 27_loops_range_function_my.py [script]

Script: save and run; predict the output first, then compare.

```
print('My name is')
i = 0
while i < 5:
    print('Jimmy Five Times (' + str(i) + ')')
    i = i + 1
```

28/37 28_loops_range_function_2.py [script]

Script: save and run; predict the output first, then compare.

```
for i in range(12, 16):  
    print(i)
```

29/37 29_loops_range_function_3.py [script]

Script: save and run; predict the output first, then compare.

```
12  
13  
14  
15
```

30/37 30_loops_range_function_4.py [script]

Script: save and run; predict the output first, then compare.

```
for i in range(0, 10, 2):  
    print(i)
```

31/37 31_loops_range_function_5.py [script]

Script: save and run; predict the output first, then compare.

```
0
2
4
6
8
```

32/37 32_loops_range_function_6.py [script]

Script: save and run; predict the output first, then compare.

```
for i in range(5, -1, -1):  
    print(i)
```

33/37 33_loops_range_function_7.py [script]

Script: save and run; predict the output first, then compare.

```
5
4
3
2
1
0
```

Topic: Importing Modules

34/37 34_importing_modules.py [script]

Script: save and run; predict the output first, then compare.

```
import random
for i in range(5):
    print(random.randint(1, 10))
```

35/37 35_importing_modules_2.py [script]

Script: save and run; predict the output first, then compare.

```
4
1
8
4
1
```

36/37 36_importing_modules_random.py [script]

Script: save and run; predict the output first, then compare.

```
import random, sys, os, math
```

Topic: Ending a Program Early with sys.exit()

37/37 37_ending_program_early.py [script]

Script: save and run; predict the output first, then compare.

```
import sys

while True:
    print('Type exit to exit.')
    response = input()
    if response == 'exit':
        sys.exit()
    print('You typed ' + response + '.')
```